CS327: Compilers

Lexical Analyzer Report

Language Name: ***OPEX***

Version 1.0

Group Members:

Gaurav Budhwani (22110085)

John Twipraham Debbarma (22110108)

Harshit (22110095)

Instructor: Prof. Shouvick Mondal

February 19, 2026

Contents

1 Introduction **3**

2 Theoretical Background **3**

2.1 What is Lexical Analysis? . . . 3

2.2 How Flex Works . . . 3

2.3 Relation to the Python Lexical Model . . . 4

3 Regular Expression Patterns **4**

4 Token Classification **5**

5 Source Code **7**

5.1 Flex Specification — `src/lexer.l` . . . 7

5.1.1 Section-by-Section Explanation . . . 8

5.2 Build Configuration — `Makefile` . . . 9

5.2.1 Explanation . . . 9

6 Test Cases **10**

6.1 Test 1: Sample Program (`sample.opx`) . . . 10

6.2 Test 2: Prime Number Checker (`prime.opx`) . . . 11

6.3 Test 3: Prime Factorisation (`prime_factorization.opx`) . . . 12

6.4 Test 4: Fibonacci Sequence — Recursive (`fibonacci.opx`) . . . 12

6.5 Test 5: Binary Search (`binary_search.opx`) . . . 13

6.6 Test 6: GCD — Recursive (`gcd_recursive.opx`) . . . 14

6.7 Test 7: Bubble Sort (`bubble_sort.opx`) . . . 15

6.8 Test 8: Palindrome Check (`palindrome.opx`) . . . 16

6.9 Test 9: Graph DFS (`dfs.opx`) . . . 17

6.10 Test 10: Binary Tree In-Order Traversal (`bst_inorder.opx`) . . . 18

6.11 Test 11: Matrix Multiplication (`matrix_mult.opx`) 19
6.12 Test 12: Edge Detection Pipeline (`edge_detection.opx`) 20

7 Token Frequency Analysis **21**

8 Conclusion **22**

1 Introduction

This report documents the design and implementation of the **Lexical Analyzer (Lexer)** for the **OPEX** programming language. The lexer is the very first stage of a compiler or interpreter pipeline: it reads raw source text character by character and groups those characters into *tokens* — the meaningful atomic units that the subsequent parser will consume.

As the Python Language Reference describes, *"a Python program is read by a parser. Input to the parser is a stream of tokens, generated by the lexical analyzer (also known as the tokenizer)"* [1], the same two-phase principle (lexing then parsing) applies to OPEX.

Our lexer is built with **Flex** (Fast Lexical Analyzer), a widely-used tool that converts a set of regular-expression rules into a deterministic finite automaton (DFA). The resulting C program reads an OPEX source file and prints one token per line, labelled with its token class (e.g. `KW_FLOW`, `ID`, `RELOP`).

The repository is structured as follows:

- `src/lexer.l` — the Flex specification containing all token rules.
- `Makefile` — the build script that runs Flex and compiles the generated C code.
- `tests/*.opx` — twelve OPEX source files that exercise different language features.
- `language_specifications.pdf` — the formal OPEX language specification.

2 Theoretical Background

2.1 What is Lexical Analysis?

Lexical analysis is the process of converting a sequence of characters into a sequence of tokens. A token is a string with an assigned and identified meaning, represented by a *token class* (also called a *token type*) and a *lexeme* (the actual text matched). For example, the text `WHILE` is the lexeme and `KW_LOOP` is its token class in OPEX.

The lexical analyser operates on the principle that source programs, while intended for human readability, must be converted into a uniform, structured stream before any syntactic or semantic analysis can occur. This phase is deliberately separated from parsing because regular expressions (and the finite automata that recognise them) are sufficient to describe all token patterns, whereas parsers require the more powerful context-free grammars.

2.2 How Flex Works

A Flex specification file (with extension `.l`) is divided into three sections separated by `%%`:

1. **Definitions**: Named regular-expression macros (e.g., `DIGIT`, `ID`) and `%option` directives.
2. **Rules**: A list of pattern–action pairs. Each pattern is a regular expression; the action is a snippet of C code executed when that pattern matches.

3. **User code**: Arbitrary C functions, including `main()`.

Flex resolves ambiguity using two principles: (a) **longest match** — it always picks the longest possible string that matches any pattern; and (b) **priority** — when two patterns match a string of equal length, the one listed first in the `.l` file wins. This is why all keywords are listed *before* the `ID` rule: `WHILE` would otherwise be tokenised as an identifier.

2.3 Relation to the Python Lexical Model

The Python lexical reference [1] provides a useful comparative lens. Python's tokeniser also handles whitespace (including significant indentation), comments (lines beginning with `#`), string literals with escape sequences, numeric literals (integer, float, etc.), and identifiers/keywords in a priority-ordered fashion. OPEX mirrors many of these categories but adds domain-specific token classes for computer-vision operations (`TYPE_CV`), convolution (`CONV_OP`), and pipeline composition (`PIPE_OP`) that have no Python equivalent.

The Python grammar specification [2] defines every construct of the language using PEG (Parsing Expression Grammar) notation, where each rule either matches or fails, with no ambiguity. When our OPEX lexer feeds its token stream to the parser in Assignment 3 (A3), a similar formal grammar will govern how those tokens combine into statements, expressions, and declarations.

3 Regular Expression Patterns

The following table summarises every named macro defined in the `Definitions` section of `lexer.l` and explains the class of strings each one matches.

Macro	Pattern	Description
`DIGIT`	`[0-9]`	Any single decimal digit, 0 through 9.
`LETTER`	`[a-zA-Z]`	Any single ASCII letter, upper or lower case.
`ID`	`{LETTER}({LETTER}|{DIGIT}|_)*`	An identifier: starts with a letter, followed by zero or more letters, digits, or underscores. Analogous to Python's `NAME` token.
`NUMBER`	`{DIGIT}+(\.{DIGIT}+)?`	An integer or fixed-point decimal literal. The fractional part is optional.
`STRING`	`"([^"\\n]|\\.)*"`	A double-quoted string literal that allows backslash escape sequences inside but forbids unescaped newlines — the same constraint Python imposes on single-line string literals [1].

Table 1: Named regular-expression macros in `lexer.l`

In addition, the following inline regular expressions appear directly in the rules section for operators and separators:

- **Assignment:** `:=` — the OPEX assignment operator, distinct from `==` (equality), which prevents the common C-style bug of writing `=` in a conditional.
- **Compound Assignment:** `+=`, `-=`, `*=`, `/=` — shorthand arithmetic-assignment operators.
- **Relational:** `==`, `!=`, `<=`, `>=`, `<`, `>` — the six comparison operators, emitting `RELOP`.
- **Increment / Decrement:** `++`, `--` — post/pre-increment operators.
- **Range:** `..` — used in `FOR` loop bounds, analogous to Python's `range()` concept.
- **Convolution:** `**` — applies a `KERNEL` to an image using spatial convolution. Please note that in Python, `**` is the exponentiation operator [1]; OPEX repurposes this symbol for image processing.
- **Pipe:** `|>` — a functional pipeline operator that passes the result of one operation as the input to the next, enabling concise image-processing chains.
- **Separators:** `[`, `]`, `(`, `)`, `:`, `,`, `.`, `;` — structural delimiters.

4 Token Classification

The table below maps every token class emitted by the lexer to its role in the OPEX language.

Token Class	Role	Example Lexemes
KW_PROG	Program structure delimiters	PROGRAM, BEGIN, END
KW_FUNC	Function-related keywords	FUNCTION, RETURN, MAIN
KW_COMPOSITE	Composite-type keywords	STRUCT, ENDSTRUCT
KW_DECL	Declaration keywords	VAR, AUTO, ARRAY, OF
TYPE	Primitive type names	INT, FLOAT, BOOL, STRING, VOID
TYPE_CV	Computer-vision type names	IMAGE, MATRIX, KERNEL, PIXEL
KW_FLOW	Conditional and match keywords	IF, THEN, ELSE, ENDIF, MATCH, WHEN, ENDMATCH
KW_LOOP	Loop keywords	FOR, FOREACH, IN, WHILE, ENDWHILE
KW_LOOP_CTRL	Loop-control keywords	TO, DO, ENDFOR, STEP
KW_IO	I/O keywords	PRINT, INPUT
KW_IMG_IO	Image file I/O	LOAD, SAVE
KW_IMG_OP	Image processing operations	GRAYSCALE, BLUR, THRESHOLD, CANNY, RESIZE, ROTATE, ERODE, DILATE, FLIP, HSV
BOOL_VAL	Boolean literals	TRUE, FALSE
LOGIC_OP	Logical operators	AND, OR, NOT
ASSIGN	Assignment operator	:=
RELOP	Relational / comparison operators	==, !=, <=, >=, <, >
ADD_OP	Additive operators	+, -
MULT_OP	Multiplicative operators	*, /, %
INC_DEC	Increment / decrement	++, --
PLUS_ASSIGN	Compound addition assignment	+=
MINUS_ASSIGN	Compound subtraction assignment	-=
MULT_ASSIGN	Compound multiplication assignment	*=
DIV_ASSIGN	Compound division assignment	/=
RANGE_OP	Range operator	..
CONV_OP	Convolution operator	**
PIPE_OP	Pipeline operator	|>
SEP	Separators and delimiters	[ ] ( ) : , . ;
ID	User-defined identifiers	n, isPrime, sobelX
INT_LIT	Integer literals	0, 42, 9999
FLOAT_LIT	Floating-point literals	3.14, 0.5
HEX_INT	Hexadecimal integer literals	0xFF
STR	String literals	"hello"
ERR	Unrecognised character (error)	any invalid character

Table 2: Complete OPEX token class reference

5 Source Code

5.1 Flex Specification — `src/lexer.l`

The file `lexer.l` is the heart of the lexer. It is divided into three sections as required by Flex.

```
%option noyywrap case-sensitive
%{
#include <stdio.h>
/* In A3, we will include "y.tab.h" here */
int line_num = 1;
%}

/* DEFINITIONS */
DIGIT       [0-9]
LETTER      [a-zA-Z]
ID          {LETTER}({LETTER}|{DIGIT}|_)*
NUMBER      {DIGIT}+(\.{DIGIT}+)?
STRING      \"([^\"\\\n]|\\.)*\"
HEXINT      0[xX][0-9a-fA-F]+
FLOAT       [0-9]+\.[0-9]+([eE][+-]?[0-9]+)?
INTEGER     [0-9]+

%%
"//".* { /* Ignore Single line */ }
"/*"([^*]|\*+[^*/])*\*+"/"  { /* Ignore Block comment */ }
[ \t\r]                     { /* Ignore whitespace */ }
\n                          { line_num++; }

    /* KEYWORDS */
"PROGRAM"|"BEGIN"|"END"     { printf("KW_PROG: %s\n", yytext); }
"FUNCTION"|"RETURN"|"MAIN"  { printf("KW_FUNC: %s\n", yytext); }
"STRUCT"|"ENDSTRUCT"        { printf("KW_COMPOSITE: %s\n", yytext); }
"VAR"|"AUTO"|"ARRAY"|"OF"   { printf("KW_DECL: %s\n", yytext); }
"INT"|"FLOAT"|"BOOL"|"STRING"|"VOID" { printf("TYPE: %s\n", yytext); }
"IMAGE"|"MATRIX"|"KERNEL"|"PIXEL"    { printf("TYPE_CV: %s\n", yytext);
    }
"MATCH"                     { printf("KW_FLOW: %s\n", yytext); }
"WHEN"                      { printf("KW_FLOW: %s\n", yytext); }
"ENDMATCH"                  { printf("KW_FLOW: %s\n", yytext); }
"IF"|"THEN"|"ELSE"|"ENDIF"  { printf("KW_FLOW: %s\n", yytext); }
"FOR"|"FOREACH"|"IN"        { printf("KW_LOOP: %s\n", yytext); }
"PRINT"|"INPUT"             { printf("KW_IO: %s\n", yytext); }
"TRUE"|"FALSE"              { printf("BOOL_VAL: %s\n", yytext); }
"AND"|"OR"|"NOT"            { printf("LOGIC_OP: %s\n", yytext); }
"TO"|"DO"|"ENDFOR"|"STEP"   { printf("KW_LOOP_CTRL: %s\n", yytext); }
"WHILE"                     { printf("KW_LOOP: %s\n", yytext); }
"ENDWHILE"                  { printf("KW_LOOP: %s\n", yytext); }

"LOAD"|"SAVE"               { printf("KW_IMG_IO: %s\n", yytext); }
"GRAYSCALE"|"HSV"           { printf("KW_IMG_OP: %s\n", yytext); }
"BLUR"|"THRESHOLD"|"CANNY"  { printf("KW_IMG_OP: %s\n", yytext); }
"RESIZE"|"ROTATE"           { printf("KW_IMG_OP: %s\n", yytext); }
"ERODE"|"DILATE"|"FLIP"     { printf("KW_IMG_OP: %s\n", yytext); }

    /* OPERATORS */
":="        { printf("ASSIGN: %s\n", yytext); }
```

```
51 ".."          { printf("RANGE_OP: %s\n", yytext); }
52 "++"|"--"     { printf("INC_DEC: %s\n", yytext); }
53 "+="                          { printf("PLUS_ASSIGN: %s\n", yytext); }
54 "-="                          { printf("MINUS_ASSIGN: %s\n", yytext); }
55 "*="                          { printf("MULT_ASSIGN: %s\n", yytext); }
56 "/="                          { printf("DIV_ASSIGN: %s\n", yytext); }
57
58 "**"                          { printf("CONV_OP: %s\n", yytext); }
59 "|>"                          { printf("PIPE_OP: %s\n", yytext); }
60 "+"|"-"       { printf("ADD_OP: %s\n", yytext); }
61 "*"|"/"|"%"  { printf("MULT_OP: %s\n", yytext); }
62 "=="|"!="|"<="|">="|"<"|">" { printf("RELOP: %s\n", yytext); }
63
64
65     /* SEPARATORS */
66 "["|"]"|"("|")"|":"|","|"."|";" { printf("SEP: %s\n", yytext); }
67
68 {ID}          { printf("ID: %s\n", yytext); }
69 {HEXINT}      { printf("HEX_INT: %s\n", yytext); }
70 {FLOAT}       { printf("FLOAT_LIT: %s\n", yytext); }
71 {INTEGER}     { printf("INT_LIT: %s\n", yytext); }
72 {STRING}      { printf("STR: %s\n", yytext); }
73
74 .             { printf("ERR: %s line %d\n", yytext, line_num); }
75 %%
76
77 int main() { yylex(); return 0; }
```

Listing 1: Full Flex specification for the OPEX lexer (lexer.l)

5.1.1 Section-by-Section Explanation

Header (`%option` and `%{ %}`) The `%option noyywrap` directive tells Flex not to call `yywrap()` at end-of-file, which is appropriate for single-file input. `case-sensitive` ensures that keywords like `WHILE` are distinct from any hypothetical `while`. The `%{ %}` block is copied verbatim into the generated C file, where we declare the `line_num` counter used by the error handler to report the offending line.

Definitions Section Five macros are defined: `DIGIT`, `LETTER`, `ID`, `NUMBER`, and `STRING`. Composing complex patterns from named primitives — rather than repeating the raw regex inline — makes the rules easier to read, maintain, and verify, much like defining named character classes in a grammar specification [2].

Rules Section — Whitespace and Comments Whitespace characters (`[ \t\r]`) are silently discarded. Newlines increment `line_num` for error reporting. Single-line comments (`"//".*`) and block comments (`/* ...*/`) are swallowed completely. This mirrors Python's treatment of comments, which the reference describes as tokens that "are ignored by the syntax" [1].

Rules Section — Keywords All reserved words are matched as string literals enclosed in double quotes. Because Flex applies the longest-match rule and keywords appear before the `{ID}` rule, a keyword like `ENDFOR` will never be incorrectly tokenised as the identifier `ENDFOR`. This is the same mechanism Python uses to reserve its keywords list:

the tokeniser checks against a fixed set before classifying a name as a general identifier [1].

Rules Section — Operators and Separators Multi-character operators (`:=`, `++`, `==`, etc.) are listed before single-character ones (`+`, `-`, `*`) to exploit the longest-match rule correctly. For example, if `++` were not listed before `+`, Flex would never match `++` as a single token — instead, it would emit two consecutive `ADD_OP: +` tokens.

Rules Section — Literals and Identifiers The {`ID`} rule catches all identifiers that were not already consumed by a keyword rule. The {`NUMBER`} rule matches integers and decimals. The {`STRING`} rule accepts double-quoted strings with backslash escapes, consistent with Python's formal grammar for string literals that allows `\"` inside a string body [1].

Error Fallthrough The final rule `.` matches any single character not handled above and prints an `ERR` message including the line number. This ensures the lexer is *total*: it never silently skips an unexpected character, which is essential for providing useful diagnostics.

5.2 Build Configuration — `Makefile`

```
all: lexer

lexer: lex.yy.c
  gcc lex.yy.c -o lexer

lex.yy.c: lexer.l
  flex lexer.l

test: lexer
  ./lexer < tests/sample.opx

clean:
  rm -f lex.yy.c lexer
```

Listing 2: Makefile for building and testing the OPEX lexer

5.2.1 Explanation

The `Makefile` defines four targets in a dependency chain. The default target `all` depends on `lexer`, which depends on `lex.yy.c`, which depends on `lexer.l`. This chain ensures that:

1. `flex lexer.l` is invoked first, generating the C source file `lex.yy.c` that implements the DFA.
2. `gcc lex.yy.c -o lexer` then compiles that C file into a standalone executable named `lexer`.
3. The `test` target pipes `tests/sample.opx` into the executable via standard input (`<`), printing the token stream to the terminal.

4. `clean` removes all generated artefacts, keeping the repository tidy for fresh builds.

Running `make` at the repository root is therefore the single command needed to build the complete lexer from scratch.

6 Test Cases

Each `.opx` file in the `tests/` directory exercises a distinct subset of OPEX's lexical features. The sections below present every test program with a thorough line-by-line analysis of the language constructs and tokens involved.

6.1 Test 1: Sample Program (`sample.opx`)

The sample program is a general-purpose demonstration used to validate the most fundamental token classes in a single pass.

```
PROGRAM MatrixMagic
BEGIN
    STRUCT Pixel
        r : INT;
        g : INT;
        b : INT;
    ENDSTRUCT

    VAR img : ARRAY[3][3] OF INT;
    VAR i   : INT;
    VAR j   : INT;

    FOR i := 0 TO 2 DO
        FOR j := 0 TO 2 DO
            img[i][j] := i + j;
        ENDFOR
    ENDFOR

    PRINT "Matrix filled.";
END
```

Listing 3: sample.opx — general feature demonstration

Token Analysis: The opening `PROGRAM MatrixMagic` yields `KW_PROG: PROGRAM` followed by `ID: MatrixMagic`. The `STRUCT` block introduces `KW_COMPOSITE` tokens for `STRUCT` and `ENDSTRUCT`, with the field names `r`, `g`, `b` producing `ID` tokens and `INT` producing `TYPE` tokens. The colon separator and semicolons each produce `SEP` tokens. The nested `FOR` loops exercise `KW_LOOP`, `ASSIGN` (`:=`), `INT_LIT` (0 and 2), `KW_LOOP_CTRL` (`TO`, `DO`, `ENDFOR`), array indexing via `SEP` (`[` and `]`) tokens, and `ADD_OP: +`.

Expected Output (Snippet):

```
KW_PROG: PROGRAM
ID: MatrixMagic
KW_PROG: BEGIN
KW_COMPOSITE: STRUCT
ID: Pixel
ID: r
```

```
SEP: :
TYPE: INT
SEP: ;
...
```

6.2 Test 2: Prime Number Checker (`prime.opx`)

This program determines whether a user-supplied integer is prime, exercising boolean logic, conditional flow, and the modulo operator.

```
PROGRAM PrimeCheck
BEGIN
    VAR n : INT;
    VAR i : INT;
    VAR isPrime : BOOL;

    PRINT "Enter a number to check:";
    INPUT n;

    isPrime := TRUE;

    IF n <= 1 THEN
        isPrime := FALSE;
    ENDIF

    FOR i := 2 TO n / 2 DO
        IF (n % i) == 0 THEN
            isPrime := FALSE;
        ENDIF
    ENDFOR

    IF isPrime == TRUE THEN
        PRINT "{n} is a Prime Number";
    ELSE
        PRINT "{n} is NOT a Prime Number";
    ENDIF
END
```

Listing 4: prime.opx — primality test

Token Analysis:

Variable declarations produce `KW_DECL: VAR`, `ID` tokens for the variable names, `SEP: :`, and either `TYPE: INT` or `TYPE: BOOL`.

Input/Output keywords `PRINT` and `INPUT` produce `KW_IO` tokens. Their arguments are either `STR` (string literals) or `ID` (variable names).

Assignment `isPrime := TRUE` tokenises as `ID: isPrime`, `ASSIGN: :=`, `BOOL_VAL: TRUE`.

Conditionals: Each `IF` block exercises four `KW_FLOW` tokens: `IF`, `THEN`, `ELSE` (where applicable), and `ENDIF`. The expression `n <= 1` produces `ID: n`, `RELOP: <=`, `INT_LIT: 1`.

Modulo expression `(n % i) == 0` generates `SEP: (`, `ID: n`, `MULT_OP: %`, `ID: i`, `SEP: )`, `RELOP: ==`, `INT_LIT: 0`. The `%` character is classified under `MULT_OP` because modulo has the same precedence as multiplication and division, an approach consistent with Python's arithmetic operator hierarchy [1].

FOR loop: The expression `n / 2` in the bound `TO n / 2` demonstrates that loop bounds can be arbitrary expressions rather than simple literals, producing `ID: n`, `MULT_OP: /`, `INT_LIT: 2`.

6.3 Test 3: Prime Factorisation (`prime_factorization.opx`)

This program computes and prints every prime factor of a positive integer using nested `WHILE` loops.

```
PROGRAM PrimeFactorisation
BEGIN
    VAR n : INT;
    VAR d : INT;

    PRINT "Enter a positive integer:";
    INPUT n;
    PRINT "Prime factors of {n}:";

    WHILE (n % 2) == 0 DO
        PRINT 2;
        n := n / 2;
    ENDWHILE

    d := 3;
    WHILE (d * d) <= n DO
        WHILE (n % d) == 0 DO
            PRINT d;
            n := n / d;
        ENDWHILE
        d := d + 2;
    ENDWHILE

    IF n > 2 THEN
        PRINT n;
    ENDIF
END
```

Listing 5: prime_factorization.opx — prime factorisation with nested WHILE loops

Token Analysis:

WHILE loop: The `WHILE` keyword produces `KW_LOOP: WHILE` and `ENDWHILE` produces `KW_LOOP: ENDWHILE`. The `DO` keyword emits `KW_LOOP_CTRL: DO`.

Nested loop condition `(d * d) <= n`: The sub-expression `d * d` demonstrates the use of `MULT_OP: *` between two identifiers. The comparison produces `RELOP: <=`.

Nested loops: The inner `WHILE` loop nested inside the outer one demonstrates that the lexer tracks newlines correctly (incrementing `line_num` for each \n), so any error token deep within the nesting will still report the correct line number.

Arithmetic update `d := d + 2`: Produces `ID: d`, `ASSIGN: :=`, `ID: d`, `ADD_OP: +`, `INT_LIT: 2` — a standard register-increment pattern.

6.4 Test 4: Fibonacci Sequence — Recursive (`fibonacci.opx`)

This program defines a recursive `FUNCTION` and a separate `MAIN` block, demonstrating OPEX's global function declaration syntax.

```
PROGRAM FibRecursive
    FUNCTION fib(n : INT) : INT
    BEGIN
        IF n <= 1 THEN
            RETURN n;
        ELSE
            RETURN fib(n - 1) + fib(n - 2);
        ENDIF
    END

MAIN
BEGIN
    VAR terms : INT;
    VAR i : INT;

    PRINT "How many terms?";
    INPUT terms;

    PRINT "Fibonacci Sequence:";
    FOR i := 0 TO terms DO
        PRINT fib(i);
    ENDFOR
END
```

Listing 6: fibonacci.opx — recursive Fibonacci function

Token Analysis:

Function signature `FUNCTION fib(n : INT) : INT` tokenises as `KW_FUNC: FUNCTION`, `ID: fib`, `SEP: (`, `ID: n`, `SEP: :`, `TYPE: INT`, `SEP: )`, `SEP: :`, `TYPE: INT`. Notice that the colon appears twice: once separating the parameter name from its type, and once separating the parameter list from the return type. Both produce `SEP: :`.

RETURN statement: `RETURN n` and `RETURN fib(n - 1) + fib(n - 2)` both begin with `KW_FUNC: RETURN`. The recursive call `fib(n - 1)` produces `ID: fib`, `SEP: (`, `ID: n`, `ADD_OP: -`, `INT_LIT: 1`, `SEP: )`, illustrating that the subtraction operator `-` is correctly disambiguated from a sign prefix by Flex's stateless context.

MAIN block: The keyword `MAIN` produces `KW_FUNC: MAIN`, establishing the program's entry point similarly to how Python designates `if __name__ == "__main__"` as the entry guard [1].

6.5 Test 5: Binary Search (`binary_search.opx`)

This program implements iterative binary search over a sorted integer array.

```
PROGRAM BinarySearch
BEGIN
    VAR data : ARRAY[10] OF INT;
    VAR target : INT;
    VAR low : INT;
    VAR high : INT;
    VAR mid : INT;
    VAR found : BOOL;

    FOR low := 0 TO 9 DO
        data[low] := low * 2;
    ENDFOR

```

```
    PRINT "Enter value to search for:";
    INPUT target;

    low := 0;
    high := 9;
    found := FALSE;

    WHILE low <= high DO
        mid := (low + high) / 2;

        IF data[mid] == target THEN
            PRINT "Found at index: {mid}";
            found := TRUE;
            low := high + 1;
        ELSE
            IF data[mid] < target THEN
                low := mid + 1;
            ELSE
                high := mid - 1;
            ENDIF
        ENDIF
    ENDWHILE

    IF found == FALSE THEN
        PRINT "Value not found";
    ENDIF
END
```

Listing 7: binary_search.opx — iterative binary search

Token Analysis:

Array declaration `VAR data : ARRAY[10] OF INT` produces the sequence `KW_DECL: VAR`, `ID: data`, `SEP: :`, `KW_DECL: ARRAY`, `SEP: [`, `INT_LIT: 10`, `SEP: ]`, `KW_DECL: OF`, `TYPE: INT`. The `ARRAY`, `[size]`, and `OF` keywords together form OPEX's array-type syntax.

Array access `data[low]` tokenises as `ID: data`, `SEP: [`, `ID: low`, `SEP: ]`.

Simulated break: Because OPEX has no dedicated `BREAK` keyword in this version, the loop is terminated by assigning `low := high + 1`, which forces the `WHILE` condition `low <= high` to be false on the next evaluation. This technique produces standard assignment tokens and no special `BREAK` token.

Nested IF–ELSE: The else branch itself contains another `IF--ELSE`, producing a doubled sequence of `KW_FLOW` tokens. The lexer handles this without difficulty because it does not track nesting depth; that is the parser's concern.

6.6 Test 6: GCD — Recursive (`gcd_recursive.opx`)

This program implements the Euclidean algorithm for Greatest Common Divisor using mutual recursion.

```
PROGRAM GCDRecursive
    FUNCTION gcd(a : INT, b : INT) : INT
    BEGIN
        IF b == 0 THEN
            RETURN a;
        ELSE
            RETURN gcd(b, a % b);
```

```
        ENDIF
    END

MAIN
BEGIN
    VAR num1 : INT;
    VAR num2 : INT;
    VAR result : INT;

    PRINT "Enter first number:";
    INPUT num1;
    PRINT "Enter second number:";
    INPUT num2;

    result := gcd(num1, num2);

    PRINT "GCD of {num1} and {num2} is {result}";
END
```

Listing 8: gcd_recursive.opx — recursive Euclidean GCD

Token Analysis:

Multi-parameter function `gcd(a : INT, b : INT)`: The comma between parameters is lexed as `SEP: ,`. Each parameter follows the same `ID : TYPE` pattern as before.

Base case `IF b == 0 THEN RETURN a`: Tests equality against the integer literal 0, producing `RELOP: ==` and `INT_LIT: 0`.

Recursive call `gcd(b, a % b)`: The modulo operator `%` inside a function-call argument list again produces `MULT_OP: %`. This recursive self-call is a key example of how the lexer must correctly tokenise `gcd` as an identifier rather than a keyword in every position (function definition header and function call body alike).

Interpolated string `"GCD of {num1} and {num2} is {result}"`: The curly-brace interpolation syntax is contained within the string literal, so the entire text is consumed as a single `STR` token by the `STRING` regex. The lexer does not parse the interpolation — that is a semantic-phase concern.

6.7 Test 7: Bubble Sort (`bubble_sort.opx`)

This program sorts a five-element integer array using the Bubble Sort algorithm.

```
PROGRAM BubbleSort
BEGIN
    VAR arr : ARRAY[5] OF INT;
    VAR i : INT;
    VAR j : INT;
    VAR temp : INT;
    VAR n : INT;

    n := 5;
    arr[0] := 64;
    arr[1] := 34;
    arr[2] := 25;
    arr[3] := 12;
    arr[4] := 22;

    PRINT "Sorting array...";

```

```
    FOR i := 0 TO n - 2 DO
        FOR j := 0 TO n - i - 2 DO
            IF arr[j] > arr[j + 1] THEN
                temp := arr[j];
                arr[j] := arr[j + 1];
                arr[j + 1] := temp;
            ENDIF
        ENDFOR
    ENDFOR

    PRINT "Sorted Array:";
    FOREACH val IN arr DO
        PRINT val;
    ENDFOR
END
```

Listing 9: bubble_sort.opx — bubble sort with FOREACH

Token Analysis:

Upper bound expressions `n - 2` and `n - i - 2`: These compound arithmetic expressions in the `TO` clause demonstrate that OPEX loop bounds are full expressions, not just literals. Each subtraction produces `ADD_OP: -`.

Swap idiom using `temp`: The three-assignment swap `temp := arr[j]; arr[j] := arr[j+1]; arr[j+1] := temp` is the classic in-place swap. Each assignment begins with an `ASSIGN: :=` token.

FOREACH loop `FOREACH val IN arr`: The `FOREACH` keyword produces `KW_LOOP: FOREACH`, the loop variable `val` produces `ID: val`, `IN` produces `KW_LOOP: IN`, and the collection name `arr` produces `ID: arr`. This is OPEX's iterator-style loop, analogous to Python's `for val in arr` syntax.

6.8 Test 8: Palindrome Check (`palindrome.opx`)

This program checks whether an integer is a numeric palindrome by reversing its digits mathematically.

```
PROGRAM PalindromeCheck
BEGIN
    VAR original : INT;
    VAR reversed : INT;
    VAR remainder : INT;
    VAR temp : INT;

    PRINT "Enter an integer:";
    INPUT original;

    reversed := 0;
    temp := original;

    WHILE temp > 0 DO
        remainder := temp % 10;
        reversed := (reversed * 10) + remainder;
        temp := temp / 10;
    ENDWHILE

    IF original == reversed THEN
        PRINT "{original} is a palindrome.";
```

```
    ELSE
        PRINT "{original} is NOT a palindrome.";
    ENDIF
END
```

Listing 10: palindrome.opx — numeric palindrome checker

Token Analysis:

Digit-extraction `remainder := temp % 10`: This line extracts the last decimal digit using modulo 10. It produces `ID: remainder`, `ASSIGN: :=`, `ID: temp`, `MULT_OP: %`, `INT_LIT: 10`.

Accumulator expression `(reversed * 10) + remainder`: The parenthesised sub-expression shifts the accumulated reversed number left by one decimal place. Tokens: `SEP: (`, `ID: reversed`, `MULT_OP: *`, `INT_LIT: 10`, `SEP: )`, `ADD_OP: +`, `ID: remainder`. This demonstrates that the lexer correctly tokenises a mix of `MULT_OP` and `ADD_OP` without any confusion, even when parentheses alter grouping.

Final comparison `original == reversed`: Produces `RELOP: ==`, the equality test.

6.9 Test 9: Graph DFS (`dfs.opx`)

This program performs a depth-first search (DFS) on a graph represented as a 5x5 adjacency matrix using a recursive function.

```
PROGRAM GraphDFS
    VAR adj     : ARRAY[5][5] OF INT;
    VAR visited : ARRAY[5] OF BOOL;

    FUNCTION dfs(u : INT) : VOID
    BEGIN
        VAR v : INT;

        PRINT "Visiting Node: {u}";
        visited[u] := TRUE;

        FOR v := 0 TO 4 DO
            IF adj[u][v] == 1 AND visited[v] == FALSE THEN
                dfs(v);
            ENDIF
        ENDFOR
    END

MAIN
BEGIN
    VAR i : INT;
    VAR j : INT;

    FOR i := 0 TO 4 DO
        visited[i] := FALSE;
        FOR j := 0 TO 4 DO
            adj[i][j] := 0;
        ENDFOR
    ENDFOR

    adj[0][1] := 1; adj[1][0] := 1;
    adj[0][3] := 1; adj[3][0] := 1;
    adj[1][2] := 1; adj[2][1] := 1;
```

```
    adj[1][4] := 1; adj[4][1] := 1;
    adj[3][4] := 1; adj[4][3] := 1;

    PRINT "Starting DFS from Node 0:";
    dfs(0);
END
```

Listing 11: dfs.opx — depth-first search on adjacency matrix

Token Analysis:

Two-dimensional array declaration `ARRAY[5][5] OF INT`: Each dimension is a separate bracketed integer literal, producing two pairs of `SEP: [` / `INT_LIT: 5` / `SEP: ]` token groups.

Two-dimensional access `adj[u][v]`: The double subscript syntax produces `ID: adj`, `SEP: [`, `ID: u`, `SEP: ]`, `SEP: [`, `ID: v`, `SEP: ]`.

Compound boolean condition `adj[u][v] == 1 AND visited[v] == FALSE`: The logical conjunction uses `LOGIC_OP: AND`. Note that `AND` is not an operator symbol but a keyword, so it appears before the `ID` rule in the specification and is therefore never confused with an identifier named `AND`. This is analogous to how Python reserves `and` as a keyword rather than an operator symbol [1].

Return type VOID in `FUNCTION dfs(u : INT) : VOID`: `VOID` produces `TYPE: VOID`, confirming that the lexer recognises `VOID` as a primitive type keyword.

6.10 Test 10: Binary Tree In-Order Traversal (`bst_inorder.opx`)

This program simulates a binary tree using a flat array (heap encoding) and performs recursive in-order traversal.

```
PROGRAM TreeTraversal
    VAR tree : ARRAY[15] OF INT;

    FUNCTION inOrder(index : INT) : VOID
    BEGIN
        VAR left  : INT;
        VAR right : INT;

        IF index >= 15 OR tree[index] == -1 THEN
            RETURN;
        ENDIF

        left  := (2 * index) + 1;
        right := (2 * index) + 2;

        inOrder(left);
        PRINT tree[index];
        inOrder(right);
    END

MAIN
BEGIN
    VAR i : INT;

    FOR i := 0 TO 14 DO
        tree[i] := -1;
    ENDFOR

```

```
    tree[0] := 10;
    tree[1] := 5;
    tree[2] := 20;
    tree[3] := 3;
    tree[4] := 7;

    PRINT "In-Order Traversal:";
    inOrder(0);
END
```

Listing 12: bst_inorder.opx — array-based binary tree in-order traversal

Token Analysis:

Compound OR condition `index >= 15 OR tree[index] == -1`: The logical disjunction uses `LOGIC_OP: OR`. The value `-1` tokenises as `ADD_OP: -` followed by `INT_LIT: 1`, since the lexer is context-free and cannot determine whether `-` is a unary negation or a binary subtraction — that distinction belongs to the parser's expression grammar [2].

Heap-index arithmetic `(2 * index) + 1`: For a zero-indexed heap, the left child of node `i` is at index $2i + 1$ and the right child at $2i + 2$. The expression produces `SEP: (`, `INT_LIT: 2`, `MULT_OP: *`, `ID: index`, `SEP: )`, `ADD_OP: +`, `INT_LIT: 1`.

Bare RETURN `RETURN;`: When `RETURN` has no value following it (base case for a `VOID` function), it still tokenises normally as `KW_FUNC: RETURN` followed by `SEP: ;`.

6.11 Test 11: Matrix Multiplication (`matrix_mult.opx`)

This program multiplies two 3×3 integer matrices using the standard triple-nested loop algorithm.

```
PROGRAM MatrixMult
BEGIN
    VAR A : ARRAY[3][3] OF INT;
    VAR B : ARRAY[3][3] OF INT;
    VAR C : ARRAY[3][3] OF INT;
    VAR i : INT;
    VAR j : INT;
    VAR k : INT;

    FOR i := 0 TO 2 DO
        FOR j := 0 TO 2 DO
            IF i == j THEN
                A[i][j] := 1;
            ELSE
                A[i][j] := 0;
            ENDIF
            B[i][j] := i + j;
            C[i][j] := 0;
        ENDFOR
    ENDFOR

    PRINT "Calculating Product...";

    FOR i := 0 TO 2 DO
        FOR j := 0 TO 2 DO
            FOR k := 0 TO 2 DO
                C[i][j] := C[i][j] + (A[i][k] * B[k][j]);
            ENDFOR
```

```
        ENDFOR
    ENDFOR

    PRINT "Result Matrix C (Row 0):";
    PRINT C[0][0]; PRINT C[0][1]; PRINT C[0][2];
END
```

Listing 13: matrix_mult.opx — 3x3 matrix multiplication

Token Analysis:

Three-level nesting: The triple `FOR` loop generates three distinct sets of `KW_LOOP: FOR` / `KW_LOOP_CTRL: DO` / `KW_LOOP_CTRL: ENDFOR` token groups. The lexer handles each identically regardless of nesting depth.

Accumulation expression `C[i][j] + (A[i][k] * B[k][j])`: The dot-product accumulation combines three 2D array accesses, an addition, and a multiplication. This single expression generates approximately 22 tokens, confirming that the lexer must process complex nested expressions correctly.

Identity-matrix initialisation using `IF i == j`: The equality test `RELOP: ==` between two loop indices demonstrates that relational operators can compare non-literal expressions.

Multiple PRINT statements on one line: `PRINT C[0][0]; PRINT C[0][1]; PRINT C[0][2]` places three statements on a single physical line separated by semicolons. Since OPEX (unlike Python) does not use indentation or newlines as statement delimiters, the lexer is indifferent to whether statements share a line.

6.12 Test 12: Edge Detection Pipeline (`edge_detection.opx`)

This program is the most domain-specific test case. It uses OPEX's computer-vision token classes to perform Sobel edge detection on an image.

```
PROGRAM EdgeDetection
BEGIN
    KERNEL sobelX := [[-1, 0, 1],
                      [-2, 0, 2],
                      [-1, 0, 1]]

    KERNEL sobelY := [[-1, -2, -1],
                      [ 0,  0,  0],
                      [ 1,  2,  1]]

    AUTO img  := LOAD("input.jpg")
    AUTO gray := GRAYSCALE(img)

    AUTO edgesX := gray ** sobelX
    AUTO edgesY := gray ** sobelY

    AUTO combined := edgesX + edgesY

    AUTO edgeMap := THRESHOLD(combined, 50)

    SAVE(edgeMap, "edges.png")
    PRINT "Edge detection complete: {img.width}x{img.height}"
END
```

Listing 14: edge_detection.opx — Sobel edge detection pipeline

Token Analysis:

KERNEL declaration `KERNEL sobelX := [[...]]`: The keyword `KERNEL` produces `TYPE_CV: KERNEL`. The variable name `sobelX` produces `ID: sobelX`. The 2D literal matrix is enclosed in nested square brackets, each producing `SEP: [` / `SEP: ]` tokens with `INT_LIT` values (and `ADD_OP: -` for negative values like `-1`).

AUTO type inference: The `AUTO` keyword produces `KW_DECL: AUTO`. It is OPEX's equivalent of C++'s `auto` or Python's implicit type inference, allowing the variable's type to be inferred from the right-hand side expression.

Image I/O `LOAD("input.jpg")` and `SAVE(edgeMap, "edges.png")`: `LOAD` produces `KW_IMG_IO: LOAD` and `SAVE` produces `KW_IMG_IO: SAVE`. Their string arguments produce `STR` tokens.

Image operation calls `GRAYSCALE(img)`, `THRESHOLD(combined, 50)`: These produce `KW_IMG_OP` tokens, a class specific to OPEX with no equivalent in Python or most general-purpose languages.

Convolution operator `gray ** sobelX`: The `**` symbol produces `CONV_OP: **`. In the OPEX language specification, this operator applies a spatial convolution of the kernel over the image. This is a deliberate syntactic distinction from the arithmetic power operator that `**` represents in Python [1], chosen because convolution is the single most common operation in image-processing pipelines.

String interpolation with member access `"{img.width}x{img.height}"`: The entire string including embedded expressions is consumed as a single `STR` token by the `STRING` regex, since braces and dots are legal characters within the double-quoted body.

7 Token Frequency Analysis

The table below summarises which token classes each test program exercises most prominently, providing a quick cross-reference for verifying lexer completeness.

Test File	Primary Token Classes Exercised
`sample.opx`	`KW_COMPOSITE`, `TYPE`, `KW_DECL`, `SEP`, `KW_LOOP`
`prime.opx`	`KW_FLOW`, `RELOP`, `MULT_OP` (%), `BOOL_VAL`, `KW_IO`
`prime_factorization.opx`	`KW_LOOP` (WHILE/ENDWHILE), `ADD_OP`, `MULT_OP`
`fibonacci.opx`	`KW_FUNC`, `ASSIGN`, `ADD_OP`, recursive `ID` calls
`binary_search.opx`	`KW_DECL` (ARRAY/OF), `SEP` (`[]`), `RELOP`, nested `KW_FLOW`
`gcd_recursive.opx`	Multi-parameter `KW_FUNC`, `MULT_OP` (%), `STR` interpolation
`bubble_sort.opx`	`KW_LOOP` (FOREACH/IN), compound `ADD_OP` bounds, `ASSIGN`
`palindrome.opx`	`MULT_OP` (*), `ADD_OP` (+), `RELOP` (==), `INT_LIT`
`dfs.opx`	2D `ARRAY`, `LOGIC_OP` (AND), `TYPE` (VOID/BOOL)
`bst_inorder.opx`	`LOGIC_OP` (OR), bare `RETURN`, negative `INT_LIT`
`matrix_mult.opx`	Triple-nested `KW_LOOP`, `MULT_OP` (*), multiple `KW_IO` per line
`edge_detection.opx`	`TYPE_CV`, `KW_IMG_IO`, `KW_IMG_OP`, `CONV_OP`, `PIPE_OP`

Table 3: Token class coverage per test file

8 Conclusion

This report has documented the complete lexical analyser for the OPEX programming language. The Flex specification in `src/lexer.l` successfully tokenises the twelve provided test programs, covering all token classes required by the OPEX language: primitive and composite type names, control-flow and loop keywords, arithmetic and logical operators, domain-specific computer-vision keywords, and the novel convolution (`**`) and pipeline (`|>`) operators. The `Makefile` provides a reproducible one-command build and test workflow.

The design principles underlying our lexer — longest match, priority ordering of rules, named macros for reusable patterns, and a catch-all error rule — are the same principles employed by production lexers such as Python's tokenizer [1]. The formal grammar reference [2] further contextualises how the flat token stream produced here will feed into the context-free grammar rules of the upcoming A3 parser phase, where the token classes become the terminal symbols of the OPEX grammar.

References

[1] Python Software Foundation. *2. Lexical Analysis — Python 3 Language Reference*. Available at: `https://docs.python.org/3/reference/lexical_analysis.html`

[2] Python Software Foundation. *10. Full Grammar Specification — Python 3 Language Reference*. Available at: `https://docs.python.org/3/reference/grammar.html`

[3] SET-IITGN/CS-327-Compilers GitHub Repository. Available at: `https://github.com/SET-IITGN/CS-327-Compilers`